Enigma Focus: Senior Android Case Study

Technical Job Interview Preparation & Architectural Deep Dive

Designed for Senior Android Engineer, Mobile Architect, and Systems Lead Interviews

---

1. Project Overview & Executive Summary

Enigma Focus is a production-grade, privacy-first, zero-friction Android application engineered in modern Kotlin and Jetpack Compose. Its mission is to eliminate compulsive digital distraction and enforce mindful barriers through hardware-level GPU controls and accessibility-level system interception.

Core Engineering Value Proposition

Unlike conventional timer apps that rely on easily bypassed foreground overlays, Enigma Focus combines:

- **Instant System-Level Interception**: Real-time detection via `AccessibilityService` and 300ms watchdog loop (< 20ms response).
- **Hardware GPU Monochrome**: Direct Daltonizer display compositor control (`WRITE_SECURE_SETTINGS`).
- **Aggressive Bedtime Lockout**: Instant screen-on locking during sleep hours with strict 60s emergency cap.
- **Unix Philosophy Architecture**: Decoupled headless engines, JSON Lines logging, and full CLI IPC via Broadcast Intents.

2. System Architecture & Clean Design

The application strictly implements **Clean Architecture** and **MVI / MVVM** patterns, ensuring total separation between the headless background daemons and the reactive Jetpack Compose presentation layer.

Layer	Key Classes	Responsibilities
Presentation	`MainScreen`, `FocusScreen`, `AppsScreen`	Declarative Compose UI, StateFlow o
Domain / Core	`FocusInterceptor`, `ScheduleEvaluator`	Pure business logic, package filtering
Hardware & Sys	`GrayscaleManager`, `BlockOverlayManager`	WindowManager, Daltonizer GPU, Vib
Headless Daemon	`FocusAccessibilityService`, `ForegroundService`	Uninterruptible background watchdog
IPC / CLI	`FocusCommandReceiver`, `JsonConfigManager`	Broadcast Intents for Termux/ADB/T
Data / Local	`AppPreferences`, `FocusEventLogger`	SharedPreferences with in-memory fa

3. Key Technical Deep Dives (Interview Talking Points)

3.1. 1. Zero-Friction App Interception: Accessibility vs UsageStats

- **Why Accessibility Service?** Android's `UsageStatsManager` suffers from 1–5 second polling latency and cannot block apps proactively. `FocusAccessibilityService` receives `TYPE_WINDOW_STATE_CHANGE` and `TYPE_WINDOWS_CHANGED` within < 20 milliseconds.

- **Optimized Accessibility Overlay**: Uses `TYPE_ACCESSIBILITY_OVERLAY` with `FLAG_NOT_TOUCH_MODAL` and `FLAG_LAYOUT_IN_SCREEN`, attached directly through the system `WindowManager` hosting an isolated ComposeView tree.
- **Keyguard Coexistence**: Evaluates `KeyguardManager.isKeyguardLocked`. It waits for `ACTION_USER_PRESENT` before presenting the sleep nudge, preventing conflicts with the system lockscreen PIN/fingerprint pad.

3.2. 2. GPU Daltonizer Engine (Hardware Monochrome Mode)

```
Settings.Secure.putInt(contentResolver, "
    accessibility_display_daltonizer_enabled", 1)
Settings.Secure.putInt(contentResolver, "accessibility_display_daltonizer",
    0) // 0 = Monochrome
```

- Rather than rendering an expensive full-screen grayscale tint shader (which degrades battery and CPU), Enigma Focus instructs the SurfaceFlinger display compositor to run monochrome on hardware GPU pipelines.
- Requires `android.permission.WRITE_SECURE_SETTINGS`, grantable via ADB or Shizuku without requiring root.

3.3. 3. Aggressive Bedtime Lockout (Anti-Relapse Watchdog)

- Monitors the overnight interval (`22:30 - 06:30`).
- Enforces a mandatory 10-second mindful breathing challenge (*Inhale, Hold, Exhale*) before enabling the emergency pause.
- Limits emergency access to **strictly 60 seconds**. When the timer expires, the background 300ms watchdog immediately re-triggers the blocking overlay and sends the user Home.
- **Snooze Invalidation**: Registering a `BroadcastReceiver` for `ACTION_SCREEN_OFF` ensures that putting the phone down immediately cancels any active 1-minute grace period.

3.4. 4. Unix Philosophy in Mobile Systems

- **Declarative Configuration**: Supports full config export/import in human-readable JSON (`enigma_config.jso` with hot live-reload.
- **Composability & CLI**: Exposes 14 Broadcast Intents enabling control via terminal or Tasker:

```
adb shell am broadcast -a com.example.enigmafocus.action.START_SESSION
    --ei duration_minutes 25
adb shell am broadcast -a com.example.enigmafocus.action.GET_STATUS
```

- **Silent Telemetry in JSON Lines**: All events logged to `focus_events.jsonl` locally (zero cloud, 100% private) with automatic rotation at 5,000 lines.

4. Likely Technical Interview Questions & Model Answers

Q1: How do you prevent memory leaks when injecting ComposeViews into an AccessibilityService?

Model Answer: "In an AccessibilityService, there is no standard Activity lifecycle. To safely render Jetpack Compose in a `WindowManager` overlay, I implemented a custom `OverlayLifecycleOwner` implementing both `LifecycleOwner` and `SavedStateRegistryOwner`. When the overlay is dismissed, we explicitly dispatch `ON_PAUSE`, `ON_STOP`, `ON_DESTROY` and invoke `windowManager.removeView(composeView)`, preventing orphaned view trees and coroutine scope leaks."

Q2: How do you handle battery optimization and aggressive OEM killers (e.g., Xiaomi MIUI/HyperOS)?

Model Answer: "We utilize a dual-layer survival architecture: 1. An official `AccessibilityService`, which is granted high-priority lifecycle protection by Android's accessibility framework. 2. A persistent `ForegroundService` with `FOREGROUND_SERVICE_TYPE_SPECIAL_USE` and ongoing notification during active timer sessions. 3. `BootReceiver` listening to `ACTION_BOOT_COMPLETED` and `MY_PACKAGE_REPLACED` to restore scheduled interval listeners immediately upon boot."

Q3: How did you test concurrency and overnight interval calculations?

Model Answer: "I wrote a comprehensive test suite in JUnit 4 using multi-threaded pools (`Executors.newFixedThr` simulating 1,000 concurrent access requests across day boundaries, leap hours, and daylight savings transitions. For overnight intervals (e.g., 22:30 to 06:30), the algorithm evaluates cross-midnight minutes math ($start > end$) and maps Saturday–Sunday rollover with 100% test coverage."

5. Key Takeaways for Interviewers

- **Engineering Rigor**: 23 automated unit and stress tests passing with 100% reliability.
- **Full Bilingual Parity**: Native dynamic localization in English and Spanish across UI, notifications, and tiles.
- **Systems-Level Mastery**: Expert handling of Android IPC, SurfaceFlinger hardware settings, Accessibility APIs, and Jetpack Compose.